

Red Hat OpenShift Data Foundation 4.22

Deploying OpenShift Data Foundation using IBM Z

Instructions on deploying Red Hat OpenShift Data Foundation to use local storage on IBM Z

Last Updated: 2026-10-23

Red Hat OpenShift Data Foundation 4.22 Deploying OpenShift Data Foundation using IBM Z

Instructions on deploying Red Hat OpenShift Data Foundation to use local storage on IBM Z

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Read this document for instructions about how to install Red Hat OpenShift Data Foundation to use local storage on IBM Z. While this document refers only to IBM Z, all information in it also applies to IBM(R) LinuxONE.

Table of Contents

PREFACE ... 3

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION ... 4

PREFACE ... 5

CHAPTER 1. PREPARING TO DEPLOY OPENSHIFT DATA FOUNDATION ... 6
1.1. REQUIREMENTS FOR INSTALLING OPENSHIFT DATA FOUNDATION USING LOCAL STORAGE DEVICES 6
1.2. ENABLING CLUSTER-WIDE ENCRYPTION WITH KMS USING THE TOKEN AUTHENTICATION METHOD 7

CHAPTER 2. DEPLOY OPENSHIFT DATA FOUNDATION USING LOCAL STORAGE DEVICES ... 8
2.1. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR 8
2.2. INSTALLING LOCAL STORAGE OPERATOR 9
2.3. FINDING AVAILABLE STORAGE DEVICES (OPTIONAL) 10
2.4. MANAGING DISKS IN A KVM ENVIRONMENT 11
2.5. ENABLING DASD DEVICES 12
2.6. CREATING OPENSHIFT DATA FOUNDATION CLUSTER ON IBM Z 12

CHAPTER 3. VERIFYING OPENSHIFT DATA FOUNDATION DEPLOYMENT FOR INTERNAL-ATTACHED DEVICES MODE ... 17
3.1. VERIFYING THE STATE OF THE PODS 17
3.2. VERIFYING THE OPENSHIFT DATA FOUNDATION CLUSTER IS HEALTHY 20
3.3. VERIFYING THE MULTICLOUD OBJECT GATEWAY IS HEALTHY 20
3.4. VERIFYING THAT THE SPECIFIC STORAGE CLASSES EXIST 20

CHAPTER 4. DEPLOYING STANDALONE MULTICLOUD OBJECT GATEWAY ... 22
4.1. INSTALLING LOCAL STORAGE OPERATOR 22
4.2. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR 22
4.3. CREATING STANDALONE MULTICLOUD OBJECT GATEWAY ON IBM Z 24

CHAPTER 5. VIEW OPENSHIFT DATA FOUNDATION TOPOLOGY ... 29

CHAPTER 6. UNINSTALLING OPENSHIFT DATA FOUNDATION ... 30
6.1. UNINSTALLING OPENSHIFT DATA FOUNDATION IN INTERNAL-ATTACHED DEVICES MODE 30
6.1.1. Removing local storage operator configurations 35
6.2. REMOVING MONITORING STACK FROM OPENSHIFT DATA FOUNDATION 39
6.3. REMOVING OPENSHIFT CONTAINER PLATFORM REGISTRY FROM OPENSHIFT DATA FOUNDATION 42
6.4. REMOVING THE CLUSTER LOGGING OPERATOR FROM OPENSHIFT DATA FOUNDATION 43

PREFACE

This document describes how to deploy Red Hat OpenShift Data Foundation using IBM Z.

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION

We appreciate your input on our documentation. Do let us know how we can make it better.

To give feedback, create a Jira ticket:

Procedure

1. Log in to the Jira.
2. Click **Create** in the top navigation bar
3. Enter a descriptive title in the Summary field.
4. Enter your suggestion for improvement in the Description field. Include links to the relevant parts of the documentation.
5. Select **Documentation** in the Components field.
6. Click Create at the bottom of the dialogue.

PREFACE

Red Hat OpenShift Data Foundation supports deployment on existing Red Hat OpenShift Container Platform (RHOCP) IBM Z clusters in connected or disconnected environments along with out-of-the-box support for proxy environments.



NOTE

See the Planning your deployment guide and the Preparing to deploy OpenShift Data Foundation chapter for more information about deployment requirements.

To deploy OpenShift Data Foundation, follow the appropriate deployment process for your environment:

- Internal Attached Devices mode – Deploy using local storage devices
- External mode

Additional resources

- Planning your deployment
- Preparing to deploy OpenShift Data Foundation
- External mode

CHAPTER 1. PREPARING TO DEPLOY OPENSHIFT DATA FOUNDATION

When you deploy OpenShift Data Foundation on Red Hat OpenShift Container Platform using local storage devices, you can create internal cluster resources. This approach internally provisions base services and all applications can access additional storage classes.

Before you begin the deployment of Red Hat OpenShift Data Foundation using local storage, ensure that your resource requirements are met. See requirements for installing OpenShift Data Foundation using local storage devices.

On the external key management system (KMS),

- When the Token authentication method is selected for encryption then refer to Enabling cluster-wide encryption with the Token authentication using KMS.
- Ensure that you are using signed certificates on your Vault servers.

After you have addressed the above, follow these steps in the order given:

1. Install the Red Hat OpenShift Data Foundation Operator .
2. Install Local Storage Operator .
3. Find the available storage devices .
4. Create the OpenShift Data Foundation cluster service on IBM Z .

1.1. REQUIREMENTS FOR INSTALLING OPENSHIFT DATA FOUNDATION USING LOCAL STORAGE DEVICES

Before installing OpenShift Data Foundation using local storage devices, your cluster must meet certain requirements.

Node requirements The cluster must consist of at least three OpenShift Container Platform worker or infrastructure nodes with locally attached-storage devices on each of them.

- Each of the three selected nodes must have at least one raw block device available. OpenShift Data Foundation uses the one or more available raw block devices.



NOTE

Make sure that the devices have a unique **by-id** device name for each available raw block device.

- The devices you use must be empty, the disks must not include Physical Volumes (PVs), Volume Groups (VGs), or Logical Volumes (LVs) remaining on the disk.

For more information, see the *Resource requirements* section in the Planning guide for IBM Z and LinuxONE.

Disconnected environment requirements

For the list of packages to be mirrored for installing OpenShift Data Foundation in disconnected environments, see the Disconnected environment section in the *Planning your deployment* guide.

Additional resources

- Planning guide - Disconnected environment
- Planning guide - Resource requirements for IBM Z and LinuxONE

1.2. ENABLING CLUSTER-WIDE ENCRYPTION WITH KMS USING THE TOKEN AUTHENTICATION METHOD

You can enable the key value backend path and policy in the vault for token authentication.

Prerequisites

- Administrator access to the vault.
- A valid Red Hat OpenShift Data Foundation Advanced subscription. For more information, see the knowledgebase article on OpenShift Data Foundation subscriptions.
- Carefully, select a unique path name as the backend **path** that follows the naming convention since you cannot change it later.

Procedure

1. Enable the Key/Value (KV) backend path in the vault.
   For vault KV secret engine API, version 1:

   ```
   $ vault secrets enable -path=odf kv
   ```

   For vault KV secret engine API, version 2:

   ```
   $ vault secrets enable -path=odf kv-v2
   ```

2. Create a policy to restrict the users to perform a write or delete operation on the secret:

   ```
   echo '
   path "odf/*" {
     capabilities = ["create", "read", "update", "delete", "list"]
   }
   path "sys/mounts" {
   capabilities = ["read"]
   }'| vault policy write odf -
   ```

3. Create a token that matches the above policy:

   ```
   $ vault token create -policy=odf -format json
   ```

CHAPTER 2. DEPLOY OPENSHIFT DATA FOUNDATION USING LOCAL STORAGE DEVICES

Deploying OpenShift Data Foundation on Red Hat OpenShift using local storage devices provides you with the option to create internal cluster resources. Follow this deployment method to use local storage to back persistent volumes for your Red Hat OpenShift applications.

Use this section to deploy OpenShift Data Foundation on IBM Z infrastructure where Red Hat OpenShift is already installed.

2.1. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR

You can install Red Hat OpenShift Data Foundation Operator using the Red Hat OpenShift Container Platform Software Catalog.

Prerequisites

- Access to an OpenShift Container Platform cluster using an account with **cluster-admin** and operator installation permissions.
- You must have at least three worker or infrastructure nodes in the Red Hat OpenShift Container Platform cluster.
- For additional resource requirements, see the Planning your deployment guide.



IMPORTANT

- When you need to override the cluster-wide default node selector for OpenShift Data Foundation, you can use the following command to specify a blank node selector for the **openshift-storage** namespace (create **openshift-storage** namespace in this case):

```
$ oc annotate namespace openshift-storage openshift.io/node-selector=
```

- Taint a node as **infra** to ensure only Red Hat OpenShift Data Foundation resources are scheduled on that node. This helps you save on subscription costs. For more information, see the *How to use dedicated worker nodes for Red Hat OpenShift Data Foundation* section in the Managing and Allocating Storage Resources guide.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**
3. Scroll or type **OpenShift Data Foundation** into the **Filter by keyword** box to find the **OpenShift Data Foundation** Operator.
4. Click **Install**.
5. Set the following options on the **Install Operator** page:

a. Update Channel as **stable-4.22**.

b. Installation Mode as **A specific namespace on the cluster**.

c. Installed Namespace as **Operator recommended namespace openshift-storage**. If Namespace **openshift-storage** does not exist, it is created during the operator installation.

d. Select Approval Strategy as **Automatic** or **Manual**.
If you select **Automatic** updates, then the Operator Lifecycle Manager (OLM) automatically upgrades the running instance of your Operator without any intervention.

If you select **Manual** updates, then the OLM creates an update request. As a cluster administrator, you must then manually approve that update request to update the Operator to a newer version.

e. Ensure that the **Enable** option is selected for the **Console plugin**.

f. Click **Install**.

Verification

- After the operator is successfully installed, the web console automatically reloads to apply the changes. During this process, a temporary error message might appear on the page and this is expected and disappears after the refresh completes.
- In the Web Console:
  - Navigate to Installed Operators and verify that the **OpenShift Data Foundation** Operator shows a green tick indicating successful installation.
  - Navigate to **Storage** and verify if the **Data Foundation** dashboard is available.

2.2. INSTALLING LOCAL STORAGE OPERATOR

Install the Local Storage Operator from the Software Catalog before creating Red Hat OpenShift Data Foundation clusters on local storage devices.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**
3. Type **local storage** in the **Filter by keyword** box to find the **Local Storage Operator** from the list of operators, and click on it.
4. Set the following options on the **Install Operator** page:
   a. Update channel as **stable**.
   b. Installation mode as **A specific namespace on the cluster**.
   c. Installed Namespace as **Operator recommended namespace openshift-local-storage**.
   d. Update approval as **Automatic**.

5. Click **Install**.

Verification

- Verify that the Local Storage Operator shows a green tick indicating successful installation.

2.3. FINDING AVAILABLE STORAGE DEVICES (OPTIONAL)

Use this procedure to identify the device names for each of the three or more worker nodes that you have labeled with the OpenShift Data Foundation label **cluster.ocs.openshift.io/openshift-storage="** before creating Persistent Volumes (PV) for IBM Z.

This step is additional information and can be skipped as the disks are automatically discovered during storage cluster creation.

Procedure

1. List and verify the name of the worker nodes with the OpenShift Data Foundation label.

```
$ oc get nodes -l=cluster.ocs.openshift.io/openshift-storage=
```

Example output:

```
NAME         STATUS   ROLES    AGE     VERSION
bmworker01   Ready    worker   6h45m   v1.16.2
bmworker02   Ready    worker   6h45m   v1.16.2
bmworker03   Ready    worker   6h45m   v1.16.2
```

2. Log in to each worker node that is used for OpenShift Data Foundation resources and find the unique **by-id** device name for each available raw block device.

```
$ oc debug node/<node name>
```

Example output:

```
$ oc debug node/bmworker01
Starting pod/bmworker01-debug ...
To use host binaries, run `chroot /host`
Pod IP: 10.0.135.71
If you don't see a command prompt, try pressing enter.
sh-4.2# chroot /host
sh-4.4# lsblk
NAME                         MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
loop0                         7:0    0   500G  0 loop
sda                           8:0    0   120G  0 disk
|-sda1                        8:1    0   384M  0 part /boot
`-sda4                        8:4    0 119.6G  0 part
`-coreos-luks-root-nocrypt   253:0    0 119.6G  0 dm   /sysroot
sdb                           8:16   0   500G  0 disk
```

In this example, for **bmworker01**, the available local device is **sdb**.

3. Identify the unique ID for each of the devices selected in Step 2.

```
sh-4.4#ls -l /dev/disk/by-id/  | grep sdb
lrwxrwxrwx. 1 root root  9 Feb  3 16:49 scsi-360050763808104bc2800000000000259 ->
../../sdb
lrwxrwxrwx. 1 root root  9 Feb  3 16:49 scsi-SIBM_2145_00e020412f0aXX00 -> ../../sdb
lrwxrwxrwx. 1 root root  9 Feb  3 16:49 scsi-0x60050763808104bc2800000000000259 ->
../../sdb
```

In the above example, the ID for the local device **sdb**

```
scsi-0x60050763808104bc2800000000000259
```

4. Repeat the above step to identify the device ID for all the other nodes that have the storage devices to be used by OpenShift Data Foundation. See this Knowledge Base article for more details.

2.4. MANAGING DISKS IN A KVM ENVIRONMENT

Use this procedure to manage disks for KVM guests, including adding disks and configuring device properties for OpenShift Data Foundation compatibility.

Procedure

1. To add a disk to a KVM guest, run the following command:

```
virsh attach-disk <guest-name> --source <disk-path> --target vdc --persistent --serial
<unique-serial-value>
```

where:

--persistent

Specifies that the configuration survives guest restarts.

--serial

Specifies that device identifiers remain constistent across restarts.

<unique-serial-value>

Specifies a unique serial value that is assigned to the disk.

2. When solid state drives (SSD) are passed through to a KVM guest, the guest might incorrectly set the rotational flag to 1, for hard disk drive (HDD). You must manually set the rotational flag to 0 so ODF can detect and use the device. To verify the current rotational flag, run the following command:

```
lsblk -o NAME,ROTA
```

Example output:

```
sh-5.1# lsblk -o NAME,ROTA
NAME   ROTA
vda       1
|-vda3    1
`-vda4    1
vdb       1
sh-5.1#
```

-

To manually set the rotational flag, follow the steps described in How to override device rotational flags in ODF/FDF environments (IBM documentation).

2.5. ENABLING DASD DEVICES

If you are using DASD devices, you must enable them before creating an OpenShift Data Foundation cluster on IBM Z. Once the DASD devices are available to z/VM guests, complete the following steps from the compute or infrastructure node on which an OpenShift Data Foundation storage node is being installed.

Procedure

1. To enable the DASD device, run the following command:

```
sudo chzdev -e <device_bus_id>
```
(1)

Replace **<device_bus_id>** with the ID of the DASD device bus-ID, such as **0.0.b100**.

To verify the status of the DASD device you can use the the **lsdasd** and **lsblk** commands.

2. To low-level format the device and specify the disk name, run the following command:

```
sudo dasdfmt /dev/<device_name> -b 4096 -p --mode=full
```
(1)

Replace **<device_name>** with the disk name, such as **dasdb**.



IMPORTANT

The use of DASD quick-formatting Extent Space Efficient (ESE) DASD is not supported on OpenShift Data Foundation. If you are using ESE DASDs, make sure to disable quick-formatting with the **--mode=full** parameter.

3. To auto-create one partition using the whole disk, run the following command:

```
sudo fdasd -a /dev/<device_name>
```
(1)

Replace **<device_name>** with the disk name you specified in the previous step, such as **dasdb**.

Once these steps are completed, the device is available during OpenShift Data Foundation deployment as **/dev/dasdb1**.



IMPORTANT

During LocalVolumeSet creation, make sure to select only the **Part** option as device type.

Additional resources

- Commands for Linux on IBM Z

2.6. CREATING OPENSHIFT DATA FOUNDATION CLUSTER ON IBM Z

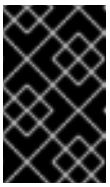
Use this procedure to create an OpenShift Data Foundation cluster on IBM Z.

Prerequisites

- Ensure that all the requirements in the Requirements for installing OpenShift Data Foundation using local storage devices section are met.
- You must have at least three worker nodes with the same storage type and size attached to each node (for example, 200 GB) to use local storage devices on IBM Z or IBM® LinuxONE.

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation → Storage Systems → Create StorageSystem**.
2. In the Backing storage page, perform the following:
   a. Select the **Create a new StorageClass using the local storage devices for Backing storage type** option.
   b. Click Next.



IMPORTANT

You are prompted to install the Local Storage Operator if it is not already installed. Click **Install**, and follow the procedure as described in Installing Local Storage Operator.

3. In the **Create local volume set** page, provide the following information:
   a. Enter a name for the **LocalVolumeSet** and the **StorageClass**.
   By default, the local volume set name appears for the storage class name. You can change the name.
   b. Choose one of the following:
      - **Disks on all nodes**
      Uses the available disks that match the selected filters on all the nodes.
      - **Disks on selected nodes**
      Uses the available disks that match the selected filters only on the selected nodes.

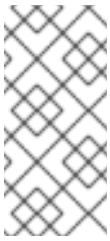


IMPORTANT

- The flexible scaling feature is enabled only when the storage cluster that you created with three or more nodes are spread across fewer than the minimum requirement of three availability zones.
  For information about flexible scaling, see knowledgebase article on *Scaling OpenShift Data Foundation cluster using YAML when flexible scaling is enabled*.
- Flexible scaling features get enabled at the time of deployment and can not be enabled or disabled later on.
- If the nodes selected do not match the OpenShift Data Foundation cluster requirement of an aggregated 30 CPUs and 72 GiB of RAM, a minimal cluster is deployed.
  For minimum starting node requirements, see the Resource requirements section in the *Planning* guide.

c. From the available list of **Disk Type**, select **SSD/NVME**.

d. Expand the **Advanced** section and set the following options:

Volume Mode	Block is selected by default.
Device Type	Select one or more device type from the dropdown list. By default, **Disk** and **Part** options are included in the **Device Type** field.  **NOTE** • For a multi-path device, select the **Mpath** option from the drop-down exclusively. • For a DASD-based cluster, ensure that only the **Part** option is included in the **Device Type** and remove the **Disk** option.
Disk Size	Set a minimum size of 100GB for the device and maximum available size of the device that needs to be included.
Maximum Disks Limit	This indicates the maximum number of PVs that can be created on a node. If this field is left empty, then PVs are created for all the available disks on the matching nodes.

e. Click **Next**.
A pop-up to confirm the creation of LocalVolumeSet is displayed.

f. Click **Yes** to continue.

4. In the **Capacity and nodes** page, configure the following:

a. **Available raw capacity** is populated with the capacity value based on all the attached disks associated with the storage class. This takes some time to show up. The **Selected nodes** list shows the nodes based on the storage class.

b. You can check the box to select **Taint** nodes.

c. Click **Next**.

5. Optional: In the **Security and network** page, configure the following based on your requirement:

a. To enable encryption, select **Enable data encryption for block and file storage**

b. Choose one or both of the following **Encryption level**:

- **Cluster-wide encryption**
  Encrypts the entire cluster (block and file).
- **StorageClass encryption**
  Creates encrypted persistent volume (block only) using encryption enabled storage class.

c. Select **Connect to an external key management service** checkbox. This is optional for cluster-wide encryption.

i. **Key Management Service Provider** is set to **Vault** by default.

ii. Enter Vault **Service Name**, host **Address** of Vault server ('https://*<hostname or ip>*''), **Port** number and **Token**.

iii. Expand **Advanced Settings** to enter additional settings and certificate details based on your Vault configuration:

A. Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.

B. Optional: Enter **TLS Server Name** and **Vault Enterprise Namespace**.

C. Upload the respective PEM encoded certificate file to provide **CA Certificate**, **Client Certificate** and **Client Private Key**.

D. Click **Save**.

d. Select **Default (SDN)** as Multus is not yet supported on OpenShift Data Foundation on IBM Z.

e. Click **Next**.

6. In the Review and create page::

a. Review the configuration details. To modify any configuration settings, click **Back** to go back to the previous configuration page.

b. Click **Create StorageSystem**.

Verification

- To verify the final Status of the installed storage cluster:

a. In the OpenShift Web Console, navigate to **Storage → Data Foundation → Storage System → ocs-storagecluster**.

b. Verify that **Status** of **StorageCluster** is **Ready** and has a green tick mark next to it.

- To verify if flexible scaling is enabled on your storage cluster, perform the following steps:

1. In the OpenShift Web Console, navigate to **Storage → Data Foundation → Storage System**.
2. From the Action menu, select **Edit Storage System**.
3. In the YAML tab, search for the keys **flexibleScaling** in **spec** section and **failureDomain** in **status** section. If **flexible scaling** is true and **failureDomain** is set to host, flexible scaling feature is enabled.

```
spec:
flexibleScaling: true
[…]
status:
failureDomain: host
```

- To verify that all components for OpenShift Data Foundation are successfully installed, see Verifying your OpenShift Data Foundation deployment .

Additional resources

- Scaling Storage guide

CHAPTER 3. VERIFYING OPENSHIFT DATA FOUNDATION DEPLOYMENT FOR INTERNAL-ATTACHED DEVICES MODE

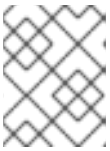
Use this section to verify that OpenShift Data Foundation is deployed correctly.

3.1. VERIFYING THE STATE OF THE PODS

Verify that all required OpenShift Data Foundation pods are in a **Running** state after deployment.

Procedure

1. Click **Workloads → Pods** from the OpenShift Web Console.
2. Select **openshift-storage** from the **Project** drop-down list.

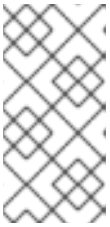


NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

For more information on the expected number of pods for each component and how it varies depending on the number of nodes, see the following table:

1. Set filter for Running and Completed pods to verify that the following pods are in **Running** and **Completed** state:



NOTE

The available pods depend on the cluster configuration. When the cluster is deployed as a standalone Multicloud Object Gateway, the **rook-ceph-operator-*** pods are not available. Similarly, when the cluster is deployed without the Multicloud Object Gateway, **noobaa-*** pods are not available.

Pods corresponding to OpenShift Data Foundation cluster

Component	Corresponding pods
OpenShift Data Foundation Operator	• **ocs-operator-*** (1 pod on any storage node) • **ocs-metrics-exporter-*** (1 pod on any storage node) • **odf-operator-controller-manager-*** (1 pod on any storage node) • **csi-addons-controller-manager-*** (1 pod on any storage node) • **odf-console-*** (1 pod on any storage node)

Component	Corresponding pods
Rook-ceph Operator	**rook-ceph-operator-*** (1 pod on any storage node)
Multicloud Object Gateway	• **noobaa-operator-*** (1 pod on any storage node) • **noobaa-core-*** (1 pod on any storage node) • **noobaa-db-pg-cluster-1** and noobaa-db-pg-cluster-2 (2 instances of MCG DB pod on any storage node) • **noobaa-endpoint-*** (1 pod on any storage node) • **cnpg-controller-manager-*** (1 pod on any storage node)
MON	**rook-ceph-mon-*** (3 pods distributed across storage nodes)
MGR	**rook-ceph-mgr-*** (2 pods on different storage nodes, one active, one standby)
MDS	**rook-ceph-mds-ocs-storagecluster-cephfilesystem-*** (2 pods distributed across storage nodes)
RGW	**rook-ceph-rgw-ocs-storagecluster-cephobjectstore-*** (1 pod on any storage node)

Component	Corresponding pods
CSI	• **cephfs** ◦ **openshift-storage.cephfs.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.cephfs.csi.ceph.com-nodeplugin-*** (1 pod on each storage node) • **nfs** ◦ **openshift-storage.nfs.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.nfs.csi.ceph.com-nodeplugin-*** (1 pod on each storage node) • **rbd** ◦ **openshift-storage.rbd.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.rbd.csi.ceph.com-nodeplugin-*** (1 pod on each storage node)
rook-ceph-crashcollector	**rook-ceph-crashcollector-*** (1 pod on each storage node)
OSD	• **rook-ceph-osd-*** (1 pod for each device) • **rook-ceph-osd-prepare-ocs-*** (1 pod for each device)
odf-blackbox-exporter	**odf-blackbox-exporter-*** (1 pod on any storage node) **NOTE:** This does not apply to clusters without storage, for example, Multicloud Object Gateway-only deployments or external mode deployments.

3.2. VERIFYING THE OPENSHIFT DATA FOUNDATION CLUSTER IS HEALTHY

Verify that the OpenShift Data Foundation cluster on IBM Z is healthy after deployment by checking storage system status and data resiliency indicators in the OpenShift Web Console.

Procedure

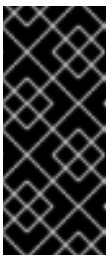
1. In the OpenShift Web Console, click **Storage → Data Foundation**.
2. Click the **Storage Systems** tab and then click on **ocs-storagecluster**.
3. In the **Status card** of Block and File dashboard under Overview tab, verify that both *Storage Cluster* and *Data Resiliency* has a green tick mark.
4. In the **Details card**, verify that the cluster information is displayed.
   For more information on the health of the OpenShift Data Foundation cluster using the Block and File dashboard, see Monitoring OpenShift Data Foundation.

3.3. VERIFYING THE MULTICLOUD OBJECT GATEWAY IS HEALTHY

Verify that the Multicloud Object Gateway is healthy after deployment to confirm that the object service and data resiliency are operational and that the NooBaa database has a recent backup.

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation**.
2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.
   a. In the **Status card** of the **Object** tab, verify that both *Object Service* and *Data Resiliency* have a green tick.
   b. In the **Details** card, verify that the MCG information is displayed.
   For more information on the health of the OpenShift Data Foundation cluster using the object service dashboard, see Monitoring OpenShift Data Foundation.



IMPORTANT

To avoid data loss, it is recommended to take a backup of NooBaa DB PVC regularly. If NooBaa DB fails and cannot be recovered, then you can revert to the latest backed-up version. For instructions on backing up your NooBaa DB, follow the steps in the knowledgebase article, Perform a One-Time Backup of the Database for the Multicloud Object Gateway.

3.4. VERIFYING THAT THE SPECIFIC STORAGE CLASSES EXIST

Verify that the expected OpenShift Data Foundation storage classes are present after deployment.

Procedure

1. Click **Storage → Storage Classes** from the left pane of the OpenShift Web Console.

2. Verify that the following storage classes are created with the OpenShift Data Foundation cluster creation:

   - **ocs-storagecluster-ceph-rbd**
   - **ocs-storagecluster-cephfs**
   - **openshift-storage.noobaa.io**
   - **ocs-storagecluster-ceph-rgw**

CHAPTER 4. DEPLOYING STANDALONE MULTICLOUD OBJECT GATEWAY

Deploying only the Multicloud Object Gateway component with the OpenShift Data Foundation provides the flexibility in deployment and helps to reduce the resource consumption. After deploying the MCG component, you can create and manage buckets using MCG object browser. For more information, see Creating and managing buckets using MCG object browser .

Use this section to deploy only the standalone Multicloud Object Gateway component, which involves the following steps:

- Installing the Local Storage Operator.
- Installing Red Hat OpenShift Data Foundation Operator
- Creating standalone Multicloud Object Gateway

4.1. INSTALLING LOCAL STORAGE OPERATOR

Use this procedure to install the Local Storage Operator from the Software Catalog before creating OpenShift Data Foundation clusters on local storage devices.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**
3. Type **local storage** in the **Filter by keyword...** box to find the **Local Storage Operator** from the list of operators and select the same.
4. Set the following options on the **Install Operator** page:
    a. Update channel as **stable**.
    b. Installation Mode as **A specific namespace on the cluster**
    c. Installed Namespace as **Operator recommended namespace openshift-local-storage**.
    d. Approval Strategy as **Automatic**.
5. Click **Install**.

Verification

- Verify that the Local Storage Operator shows a green tick indicating successful installation.

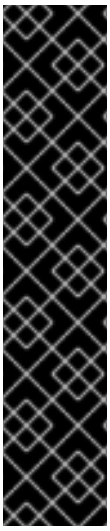
4.2. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR

You can install Red Hat OpenShift Data Foundation Operator by using the Red Hat OpenShift Container Platform Software Catalog.

For information about the hardware and software requirements, see Planning your deployment.

Prerequisites

- Access to an OpenShift Container Platform cluster using an account with **cluster-admin** and Operator installation permissions.
- You must have at least three worker nodes in the Red Hat OpenShift Container Platform cluster.



IMPORTANT

- When you need to override the cluster-wide default node selector for OpenShift Data Foundation, you can use the following command in the command line interface to specify a blank node selector for the **openshift-storage** namespace (create openshift-storage namespace in this case):

```
$ oc annotate namespace openshift-storage openshift.io/node-selector=
```

- Taint a node as **infra** to ensure only Red Hat OpenShift Data Foundation resources are scheduled on that node. This helps you save on subscription costs. For more information, see How to use dedicated worker nodes for Red Hat OpenShift Data Foundation chapter in the Managing and Allocating Storage Resources guide.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**
3. Scroll or type **OpenShift Data Foundation** into the **Filter by keyword** box to find the **OpenShift Data Foundation** Operator.
4. Click **Install**.
5. Set the following options on the **Install Operator** page:
   a. Update Channel as **stable-4.22**.
   b. Installation Mode as **A specific namespace on the cluster**
   c. Installed Namespace as **Operator recommended namespace openshift-storage**. If Namespace **openshift-storage** does not exist, it is created during the operator installation.
6. Select **Approval Strategy** as **Automatic** or **Manual**.
   If you select **Automatic** updates, then the Operator Lifecycle Manager (OLM) automatically upgrades the running instance of your Operator without any intervention.

   If you select **Manual** updates, then the OLM creates an update request. As a cluster administrator, you must then manually approve that update request to update the Operator to a newer version.
7. Ensure that the **Enable** option is selected for the **Console plugin**.
8. Click **Install**.

Verification

Verification

- Verify that the **OpenShift Data Foundation** Operator shows a green tick indicating successful installation.
- After the operator is successfully installed, the web console automatically reloads to apply the changes. During this process, a temporary error message might appear on the page and this is expected and disappears after the refresh completes.
    - In the Web Console, navigate to **Storage** and verify if **Data Foundation** is available.

4.3. CREATING STANDALONE MULTICLOUD OBJECT GATEWAY ON IBM Z

You can create only the standalone Multicloud Object Gateway component while deploying OpenShift Data Foundation.

Prerequisites

- Ensure that the OpenShift Data Foundation Operator is installed.
- The Multicloud Object Gateway database uses ReadWriteOncePod (RWOP) access mode to safeguard against node failures. If the underlying CSI driver does not support RWOP, the access mode must be manually changed to ReadWriteOnce (RWO) before deployment. For more information, see the knowledgebase article How to change NooBaa DB access mode from RWOP to RWO in standalone MCG deployments?.
- (For deploying using local storage devices only) Ensure that Local Storage Operator is installed.

To identify storage devices on each node, see Finding available storage devices.

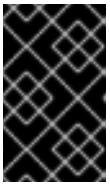
Procedure

1. Log into the OpenShift Web Console.
2. In **openshift-local-storage** namespace, click **Ecosystem → Installed Operators** to view the installed operators.
3. Click the **Local Storage** installed operator.
4. On the **Operator Details** page, click the **Local Volume** link.
5. Click **Create Local Volume**.
6. Click on **YAML view** for configuring Local Volume.
7. Define a **LocalVolume** custom resource for filesystem PVs using the following YAML.

```
apiVersion: local.storage.openshift.io/v1
kind: LocalVolume
metadata:
  name: localblock
  namespace: openshift-local-storage
spec:
  logLevel: Normal
  managementState: Managed
  nodeSelector:
```

```
    nodeSelectorTerms:
     - matchExpressions:
         - key: kubernetes.io/hostname
           operator: In
           values:
             - worker-0
             - worker-1
             - worker-2
  storageClassDevices:
    - devicePaths:
        - /dev/sda
      storageClassName: localblock
      volumeMode: Filesystem
```

The above definition selects **sda** local device from the **worker-0**, **worker-1** and **worker-2** nodes. The **localblock** storage class is created and persistent volumes are provisioned from **sda**.



IMPORTANT

Specify appropriate values of nodeSelector as per your environment. The device name should be same on all the worker nodes. You can also specify more than one devicePaths.

8. Click **Create**.
9. In the OpenShift Web Console, navigate to **Storage → StorageCluster**.
10. On the Welcome page, click **Configure Data Foundation**.
11. Select **Setup Multicloud Object Gateway**.
12. Select the **Use an existing StorageClass** option for **Backing storage type**.
    a. Select the **Storage Class** that you used while installing LocalVolume.
13. Click **Next**.
14. Optional: In the **Security** page, select the **Connect to an external key management service** checkbox. This is optional for cluster-wide encryption.
    a. From the **Key Management Service Provider** drop down list, either select **Vault** or **Thales CipherTrust Manager (using KMIP)**. If you selected **Vault**, go to the next step. If you selected **Thales CipherTrust Manager (using KMIP)**, go to step iii.
    b. Select an **Authentication Method**.

    Using Token authentication method

    - Enter a unique **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Token**.
    - Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:
        - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.

- Optional: Enter **TLS Server Name** and **Vault Enterprise Namespace**.
- Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key**.
- Click **Save** and skip to step iv.

Using Kubernetes authentication method

- Enter a unique Vault **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Role** name.
- Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:
  - Enter the Key Value secret path in the **Backend Path** that is dedicated and unique to OpenShift Data Foundation.
  - Optional: Enter **TLS Server Name** and **Authentication Path** if applicable.
  - Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate**, and **Client Private Key**.
  - Click **Save** and skip to step iv.

c. To use **Thales CipherTrust Manager (using KMIP)** as the KMS provider, follow the steps below:

i. Enter a unique **Connection Name** for the Key Management service within the project.

ii. In the **Address** and **Port** sections, enter the IP of Thales CipherTrust Manager and the port where the KMIP interface is enabled. For example:

- **Address**: 123.34.3.2
- **Port**: 5696

iii. Upload the **Client Certificate**, **CA certificate**, and **Client Private Key**.

iv. If StorageClass encryption is enabled, enter the Unique Identifier to be used for encryption and decryption generated above.

v. The **TLS Server** field is optional and used when there is no DNS entry for the KMIP endpoint. For example, **kmip_all_<port>.ciphertrustmanager.local**.

d. Select a **Network**.

e. Click **Next**.

15. In the **Review and create** page, review the configuration details:
To modify any configuration settings, click **Back**.

16. Click **Create StorageSystem**.

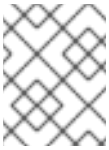
Verification

Verifying that the OpenShift Data Foundation cluster is healthy

1. In the OpenShift Web Console, click **Storage → Data Foundation**.
2. Click the **Storage Systems** tab and then click on **ocs-storagecluster**.
   a. In the **Status card** of the **Object** tab, verify that both *Object Service* and *Data Resiliency* have a green tick.
   b. In the **Details** card, verify that the MCG information is displayed.

Verifying the state of the pods

1. Click **Workloads → Pods** from the OpenShift Web Console.
2. Select **openshift-storage** from the **Project** drop-down list and verify that the following pods are in **Running** state.



NOTE

If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

Component	Corresponding pods
OpenShift Data Foundation Operator	• **ocs-operator-*** (1 pod on any storage node) • **ocs-metrics-exporter-*** (1 pod on any storage node) • **odf-operator-controller-manager-*** (1 pod on any storage node) • **odf-console-*** (1 pod on any storage node) • **csi-addons-controller-manager-*** (1 pod on any storage node)
Rook-ceph Operator	**rook-ceph-operator-*** (1 pod on any storage node)

Component	Corresponding pods
Multicloud Object Gateway	• **noobaa-operator-*** (1 pod on any storage node) • **noobaa-core-*** (1 pod on any storage node) • **noobaa-db-pg-cluster-1** and noobaa-db-pg-cluster-2 (2 instances of MCG DB pod on any storage node) • **noobaa-endpoint-*** (1 pod on any storage node) • **cnpg-controller-manager-*** (1 pod on any storage node) • **noobaa-default-backing-store-noobaa-pod-*** (1 pod on any storage node)

CHAPTER 5. VIEW OPENSHIFT DATA FOUNDATION TOPOLOGY

The topology provides a visual representation of the OpenShift Data Foundation storage cluster at multiple levels of abstraction and enables interaction with each layer. It illustrates how the cluster components are organized and related, while offering visibility into Object Storage Devices (OSDs), including their count, type, and status.

Procedure

1. On the OpenShift Web Console, navigate to **Storage → Data Foundation → Topology**.
   The view shows the storage cluster and the topology groupings inside it. You can see the nodes depicted by circular entities within the topology groupings, which are indicated by dotted lines. The topology groupings are based on the cluster's topology configuration (for example, racks in bare metal or vSphere clusters, or zones in cloud-based clusters). The label of each item or resource contains basic information such as status and health or indication for alerts.

2. Choose a node to view node details on the right-hand panel. You can also access resources or deployments within a node by clicking on the search/preview decorator icon.
   The node details panel displays information about the OSDs running on that node, including:

   - Number of OSDs on the node
   - OSD disk type (for example, SSD, HDD)
   - OSD status (for example, Running, Error, CrashLoopBackOff)

3. To view deployment details

   a. Click the preview decorator on a node. A modal window appears above the node that displays all of the deployments associated with that node along with their statuses.

   b. Click the **Back to main view** button in the model's upper left corner to close and return to the previous view.

   c. Select a specific deployment to see more information about it. All relevant data is shown in the side panel.

4. Click the **Resources** tab to view the pods information. This tab provides a deeper understanding of the problems and offers granularity that aids in better troubleshooting.

5. Click the **OSD information** tab to view detailed information about the OSDs. This tab provides more comprehensive OSD-specific details, including:

   - OSD pod name and status
   - Disk device associated with each OSD
   - Device class (for example, SSD, HDD)
   - Resource utilization metrics

6. Click the pod links to view the pod information page on OpenShift Container Platform. The link opens in a new window.

CHAPTER 6. UNINSTALLING OPENSHIFT DATA FOUNDATION

6.1. UNINSTALLING OPENSHIFT DATA FOUNDATION IN INTERNAL-ATTACHED DEVICES MODE

Uninstall OpenShift Data Foundation in internal-attached devices mode from IBM Z by removing the storage cluster, PVCs, local storage configurations, and the operator in the correct order.

Uninstall Annotations

Annotations on the Storage Cluster are used to change the behavior of the uninstall process. To define the uninstall behavior, the following two annotations have been introduced in the storage cluster:

- **uninstall.ocs.openshift.io/cleanup-policy: delete**
- **uninstall.ocs.openshift.io/mode: graceful**

The following table provides information on the different values that can used with these annotations:

uninstall.ocs.openshift.io uninstall annotations descriptions

Annotation	Value	Default	Behavior
cleanup-policy	delete	Yes	Rook cleans up the physical drives and the **DataDirHostPath**
cleanup-policy	retain	No	Rook does **not** clean up the physical drives and the **DataDirHostPath**
mode	graceful	Yes	Rook and NooBaa **pauses** the uninstall process until the administrator/user removes the Persistent Volume Claims (PVCs) and Object Bucket Claims (OBCs)
mode	forced	No	Rook and NooBaa proceeds with uninstall even if the PVCs/OBCs provisioned using Rook and NooBaa exist respectively

Edit the value of the annotation to change the cleanup policy or the uninstall mode.

```
$ oc -n openshift-storage annotate storagecluster ocs-storagecluster
uninstall.ocs.openshift.io/cleanup-policy="retain" --overwrite
```

```
$ oc -n openshift-storage annotate storagecluster ocs-storagecluster
uninstall.ocs.openshift.io/mode="forced" --overwrite
```

Expected output for both commands:

```
storagecluster.ocs.openshift.io/ocs-storagecluster annotated
```

Prerequisites

- Ensure that the OpenShift Data Foundation cluster is in a healthy state. The uninstall process can fail when some of the pods are not terminated successfully due to insufficient resources or nodes. In case the cluster is in an unhealthy state, contact Red Hat Customer Support before uninstalling OpenShift Data Foundation.
- Ensure that applications are not consuming persistent volume claims (PVCs) or object bucket claims (OBCs) using the storage classes provided by OpenShift Data Foundation.
- If any custom resources (such as custom storage classes, cephblockpools) were created by the admin, they must be deleted by the admin after removing the resources which consumed them.

Procedure

1. Delete the volume snapshots that are using OpenShift Data Foundation.
   a. List the volume snapshots from all the namespaces.

      ```
      $ oc get volumesnapshot --all-namespaces
      ```

   b. From the output of the previous command, identify and delete the volume snapshots that are using OpenShift Data Foundation.

      ```
      $ oc delete volumesnapshot <VOLUME-SNAPSHOT-NAME> -n <NAMESPACE>
      ```

      <VOLUME-SNAPSHOT-NAME>

      Is the name of the volume snapshot

      <NAMESPACE>

      Is the project namespace

2. Delete PVCs and OBCs that are using OpenShift Data Foundation.
   In the default uninstall mode (graceful), the uninstaller waits till all the PVCs and OBCs that use OpenShift Data Foundation are deleted.

   If you want to delete the Storage Cluster without deleting the PVCs, you can set the uninstall mode annotation to **forced** and skip this step. Doing so results in orphan PVCs and OBCs in the system.

   a. Delete OpenShift Container Platform monitoring stack PVCs using OpenShift Data Foundation.
      See Removing monitoring stack from OpenShift Data Foundation

   b. Delete OpenShift Container Platform Registry PVCs using OpenShift Data Foundation.
      Removing OpenShift Container Platform registry from OpenShift Data Foundation

c. Delete OpenShift Container Platform logging PVCs using OpenShift Data Foundation. Removing the cluster logging operator from OpenShift Data Foundation

d. Delete the other PVCs and OBCs provisioned using OpenShift Data Foundation.

- Given below is a sample script to identify the PVCs and OBCs provisioned using OpenShift Data Foundation. The script ignores the PVCs that are used internally by OpenShift Data Foundation.

```
#!/bin/bash

RBD_PROVISIONER="openshift-storage.rbd.csi.ceph.com"
CEPHFS_PROVISIONER="openshift-storage.cephfs.csi.ceph.com"
NOOBAA_PROVISIONER="openshift-storage.noobaa.io/obc"
RGW_PROVISIONER="openshift-storage.ceph.rook.io/bucket"

NOOBAA_DB_PG_1_PVC="noobaa-db-pg-cluster-1"
NOOBAA_DB_PG_2_PVC="noobaa-db-pg-cluster-2"
NOOBAA_BACKINGSTORE_PVC="noobaa-default-backing-store-noobaa-pvc"

# Find all the OCS StorageClasses
OCS_STORAGECLASSES=$(oc get storageclasses | grep -e
"$RBD_PROVISIONER" -e "$CEPHFS_PROVISIONER" -e
"$NOOBAA_PROVISIONER" -e "$RGW_PROVISIONER" | awk '{print $1}')

# List PVCs in each of the StorageClasses
for SC in $OCS_STORAGECLASSES
do
      echo
"=============================================================================
=="
      echo "$SC StorageClass PVCs and OBCs"
      echo
"=============================================================================
=="
      oc get pvc  --all-namespaces --no-headers 2>/dev/null | grep $SC | grep -v -e
"$NOOBAA_DB_PVC" -e "$NOOBAA_BACKINGSTORE_PVC"
      oc get obc  --all-namespaces --no-headers 2>/dev/null | grep $SC
      echo
done
```



NOTE

Omit **RGW_PROVISIONER** for cloud platforms.

- Delete the OBCs.

```
$ oc delete obc <obc-name> -n <project-name>
```

<obc-name>

Is the name of the OBC

<project-name>

Is the name of the project

- Delete the PVCs.

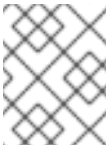
```
$ oc delete pvc <pvc-name> -n <project-name>
```

<pvc-name>

Is the name of the PVC

<project-name>

Is the name of the project



NOTE

Ensure that you have removed any custom backing stores, bucket classes, etc., created in the cluster.

3. Delete the Storage System object and wait for the removal of the associated resources.

```
$ oc delete -n openshift-storage storagesystem --all --wait=true
```

4. Check the cleanup pods if the **uninstall.ocs.openshift.io/cleanup-policy** was set to **delete**(default) and ensure that their status is **Completed**.

```
$ oc get pods -n openshift-storage | grep -i cleanup
```

Example output:

```
NAME                          READY   STATUS      RESTARTS   AGE
cluster-cleanup-job-<xx>       0/1     Completed   0          8m35s
cluster-cleanup-job-<yy>       0/1     Completed   0          8m35s
cluster-cleanup-job-<zz>       0/1     Completed   0          8m35s
```

5. Confirm that the directory **/var/lib/rook** is now empty. This directory is empty only if the **uninstall.ocs.openshift.io/cleanup-policy** annotation was set to **delete**(default).

```
$ for i in $(oc get node -l cluster.ocs.openshift.io/openshift-storage= -o jsonpath='{
.items[*].metadata.name }'); do oc debug node/${i} -- chroot /host  ls -l /var/lib/rook; done
```

6. If encryption was enabled at the time of install, remove **dm-crypt** managed **device-mapper** mapping from the OSDs on all the OpenShift Data Foundation nodes.

   a. Create a **debug** pod and **chroot** to the host on the storage node.

```
$ oc debug node/<node-name>
```

```
$ chroot /host
```

<node-name>

Is the name of the node

   b. Get Device names and make note of the OpenShift Data Foundation devices.

```
$ dmsetup ls
```

Example output:

```
ocs-deviceset-0-data-0-57snx-block-dmcrypt (253:1)
```

c. Remove the mapped device.

```
$ cryptsetup luksClose --debug --verbose ocs-deviceset-0-data-0-57snx-block-dmcrypt
```



IMPORTANT

If the above command gets stuck due to insufficient privileges, run the following commands:

- Press **CTRL+Z** to exit the above command.
- Find PID of the process which was stuck.

  ```
  $ ps -ef | grep crypt
  ```

- Terminate the process using **kill** command.

  ```
  $ kill -9 <PID>
  ```

  <PID>

  Is the process ID

- Verify that the device name is removed.

  ```
  $ dmsetup ls
  ```

7. Delete the namespace and wait till the deletion is complete. You need to switch to another project if **openshift-storage** is the active project.
   For example:

```
$ oc project default
```

```
$ oc delete project openshift-storage --wait=true --timeout=5m
```

The project is deleted if the following command returns a NotFound error.

```
$ oc get project openshift-storage
```



NOTE

While uninstalling OpenShift Data Foundation, if **namespace** is not deleted completely and remains in **Terminating** state, perform the steps in Troubleshooting and deleting remaining resources during Uninstall to identify objects that are blocking the namespace from being terminated.

8. Delete local storage operator configurations if you have deployed OpenShift Data Foundation using local storage devices. See Removing local storage operator configurations .

9. Unlabel the storage nodes.

```
$ oc label nodes  --all cluster.ocs.openshift.io/openshift-storage-
```

```
$ oc label nodes  --all topology.rook.io/rack-
```

10. Remove the OpenShift Data Foundation taint if the nodes were tainted.

```
$ oc adm taint nodes --all node.ocs.openshift.io/storage-
```

11. Confirm all the Persistent volumes (PVs) provisioned using OpenShift Data Foundation are deleted. If there is any PV left in the **Released** state, delete it.

```
$ oc get pv
```

```
$ oc delete pv <pv-name>
```

<pv-name>

Is the name of the PV

12. Remove the **CustomResourceDefinitions**.

```
$ oc delete crd backingstores.noobaa.io bucketclasses.noobaa.io
cephblockpools.ceph.rook.io cephclusters.ceph.rook.io cephfilesystems.ceph.rook.io
cephnfses.ceph.rook.io cephobjectstores.ceph.rook.io cephobjectstoreusers.ceph.rook.io
noobaas.noobaa.io ocsinitializations.ocs.openshift.io storageclusters.ocs.openshift.io
cephclients.ceph.rook.io cephobjectrealms.ceph.rook.io cephobjectzonegroups.ceph.rook.io
cephobjectzones.ceph.rook.io cephrbdmirrors.ceph.rook.io storagesystems.odf.openshift.io --
wait=true --timeout=5m
```

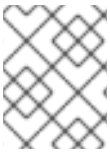
13. To ensure that OpenShift Data Foundation is uninstalled completely, on the OpenShift Container Platform Web Console,

    a. Click **Storage**.

    b. Verify that **OpenShift Data Foundation** no longer appears under Storage.

6.1.1. Removing local storage operator configurations

Remove the Local Storage Operator configurations from IBM Z nodes as part of the OpenShift Data Foundation uninstall procedure to clean up LocalVolumeSet and LocalVolume resources before removing the operator itself.



NOTE

For OpenShift Data Foundation deployments only using **localvolume** resources, go directly to step 8.

Procedure

1. Identify the **LocalVolumeSet** and the corresponding **StorageClassName** being used by OpenShift Data Foundation.

```
$ oc get localvolumesets.local.storage.openshift.io -n openshift-local-storage
```

2. Set the variable SC to the **StorageClass** providing the **LocalVolumeSet**.

```
$ export SC="<StorageClassName>"
```

3. List and note the devices to be cleaned up later. Inorder to list the device ids of the disks, follow the procedure mentioned here, See Find the available storage devices .
Example output:

```
/dev/disk/by-id/scsi-360050763808104bc28000000000000eb
/dev/disk/by-id/scsi-360050763808104bc28000000000000ef
/dev/disk/by-id/scsi-360050763808104bc28000000000000f3
```

4. Delete the **LocalVolumeSet**.

```
$ oc delete localvolumesets.local.storage.openshift.io <name-of-volumeset> -n openshift-local-storage
```

5. Delete the local storage PVs for the given **StorageClassName**.

```
$ oc get pv | grep $SC | awk '{print $1}'| xargs oc delete pv
```

6. Delete the **StorageClassName**.

```
$ oc delete sc $SC
```

7. Delete the symlinks created by the **LocalVolumeSet**.

```
[[ ! -z $SC ]] && for i in $(oc get node -l cluster.ocs.openshift.io/openshift-storage= -o jsonpath='{ .items[*].metadata.name }'); do oc debug node/${i} -- chroot /host rm -rfv /mnt/local-storage/${SC}/; done
```

8. Delete **LocalVolumeDiscovery**.

```
$ oc delete localvolumediscovery.local.storage.openshift.io/auto-discover-devices -n openshift-local-storage
```

9. Remove the **LocalVolume** resources (if any).
Use the following steps to remove the **LocalVolume** resources that were used to provision PVs in the current or previous OpenShift Data Foundation version. Also, ensure that these resources are not being used by other tenants on the cluster.

   For each of the local volumes, do the following:

   a. Identify the **LocalVolume** and the corresponding **StorageClassName** being used by OpenShift Data Foundation.

```
$ oc get localvolume.local.storage.openshift.io -n openshift-local-storage
```

b. Set the variable LV to the name of the LocalVolume and variable SC to the name of the StorageClass
For example:

```
$ LV=local-block
$ SC=localblock
```

c. List and note the devices to be cleaned up later.

```
$ oc get localvolume -n openshift-local-storage $LV -o jsonpath='{
.spec.storageClassDevices[].devicePaths[] }{"\n"}'
```

Example output:

```
/dev/sdb
/dev/sdc
/dev/sdd
/dev/sde
```

d. Delete the local volume resource.

```
$ oc delete localvolume -n openshift-local-storage --wait=true $LV
```

e. Delete the remaining PVs and StorageClasses if they exist.

```
$ oc delete pv -l storage.openshift.com/local-volume-owner-name=${LV} --wait --
timeout=5m
$ oc delete storageclass $SC --wait --timeout=5m
```

f. Clean up the artifacts from the storage nodes for that resource.

```
$ [[ ! -z $SC ]] && for i in $(oc get node -l cluster.ocs.openshift.io/openshift-storage= -o
jsonpath='{ .items[*].metadata.name }'); do oc debug node/${i} -- chroot /host rm -rfv
/mnt/local-storage/${SC}/; done
```

Example output:

```
Starting pod/node-xxx-debug ...
To use host binaries, run `chroot /host`
removed '/mnt/local-storage/localblock/nvme2n1'
removed directory '/mnt/local-storage/localblock'

Removing debug pod ...
Starting pod/node-yyy-debug ...
To use host binaries, run `chroot /host`
removed '/mnt/local-storage/localblock/nvme2n1'
removed directory '/mnt/local-storage/localblock'

Removing debug pod ...
Starting pod/node-zzz-debug ...
To use host binaries, run `chroot /host`
removed '/mnt/local-storage/localblock/nvme2n1'
```

```
removed directory '/mnt/local-storage/localblock'

Removing debug pod ...
```

10. Wipe the disks for each of the local volumesets or local volumes listed in step 1 and 8 respectively so that they can be reused.

    a. List the storage nodes.

    ```
    oc get nodes -l cluster.ocs.openshift.io/openshift-storage=
    ```

    Example output:

    ```
    NAME      STATUS   ROLES    AGE     VERSION
    node-xxx  Ready    worker   4h45m  v1.18.3+6c42de8
    node-yyy  Ready    worker   4h46m  v1.18.3+6c42de8
    node-zzz  Ready    worker   4h45m  v1.18.3+6c42de8
    ```

    b. Obtain the node console and execute **chroot /host** command when the prompt appears.

    ```
    $ oc debug node/node-xxx
    Starting pod/node-xxx-debug …
    To use host binaries, run `chroot /host`
    Pod IP: w.x.y.z
    If you don't see a command prompt, try pressing enter.
    sh-4.2# chroot /host
    ```

    c. Store the disk paths in the **DISKS** variable within quotes. For the list of disk paths, see step 3 and step 8.c for local volumeset and local volume respectively.
    Example output:

    ```
    sh-4.4# DISKS="/dev/disk/by-id/scsi-360050763808104bc28000000000000eb
    /dev/disk/by-id/scsi-360050763808104bc28000000000000ef /dev/disk/by-id/scsi-
    360050763808104bc28000000000000f3 "
    or
    sh-4.2# DISKS="/dev/sdb /dev/sdc /dev/sdd /dev/sde ".
    ```

    d. Run **sgdisk --zap-all** on all the disks.

    ```
    sh-4.4# for disk in $DISKS; do sgdisk --zap-all $disk;done
    ```

    Example output:

    ```
    Creating new GPT entries.
    GPT data structures destroyed! You may now partition the disk using fdisk or
    other utilities.
    Creating new GPT entries.
    GPT data structures destroyed! You may now partition the disk using fdisk or
    other utilities.
    Creating new GPT entries.
    GPT data structures destroyed! You may now partition the disk using fdisk or
    other utilities.
    ```

```
Creating new GPT entries.
GPT data structures destroyed! You may now partition the disk using fdisk or
other utilities.
```

e. Exit the shell and repeat for the other nodes.

```
sh-4.4# exit
exit
sh-4.2# exit
exit
Removing debug pod ...
```

11. Delete the **openshift-local-storage** namespace and wait till the deletion is complete. You will need to switch to another project if the **openshift-local-storage** namespace is the active project.
For example:

```
$ oc project default
$ oc delete project openshift-local-storage --wait=true --timeout=5m
```

The project is deleted if the following command returns a **NotFound** error.

```
$ oc get project openshift-local-storage
```

6.2. REMOVING MONITORING STACK FROM OPENSHIFT DATA FOUNDATION

Use this section to clean up the monitoring stack from OpenShift Data Foundation.

The Persistent Volume Claims (PVCs) that are created as a part of configuring the monitoring stack are in the **openshift-monitoring** namespace.

Prerequisites

- PVCs are configured to use the OpenShift Container Platform monitoring stack.
For more information, see configuring monitoring stack.

Procedure

1. List the pods and PVCs that are currently running in the **openshift-monitoring** namespace.

```
$ oc get pod,pvc -n openshift-monitoring
```

Example output:

```
NAME                        READY   STATUS    RESTARTS   AGE
pod/alertmanager-main-0     3/3     Running   0          8d
pod/alertmanager-main-1     3/3     Running   0          8d
pod/alertmanager-main-2     3/3     Running   0          8d
pod/cluster-monitoring-
operator-84457656d-pkrxm    1/1     Running   0          8d
pod/grafana-79ccf6689f-2ll28   2/2     Running   0          8d
```

```
pod/kube-state-metrics-
7d86fb966-rvd9w              3/3     Running   0          8d
pod/node-exporter-25894      2/2     Running   0          8d
pod/node-exporter-4dsd7      2/2     Running   0          8d
pod/node-exporter-6p4zc      2/2     Running   0          8d
pod/node-exporter-jbjvg      2/2     Running   0          8d
pod/node-exporter-jj4t5      2/2     Running   0          6d18h
pod/node-exporter-k856s      2/2     Running   0          6d18h
pod/node-exporter-rf8gn      2/2     Running   0          8d
pod/node-exporter-rmb5m      2/2     Running   0          6d18h
pod/node-exporter-zj7kx      2/2     Running   0          8d
pod/openshift-state-metrics-
59dbd4f654-4clng             3/3     Running   0          8d
pod/prometheus-adapter-
5df5865596-k8dzn             1/1     Running   0          7d23h
pod/prometheus-adapter-
5df5865596-n2gj9             1/1     Running   0          7d23h
pod/prometheus-k8s-0         6/6     Running   1          8d
pod/prometheus-k8s-1         6/6     Running   1          8d
pod/prometheus-operator-
55cfb858c9-c4zd9             1/1     Running   0          6d21h
pod/telemeter-client-
78fc8fc97d-2rgfp             3/3     Running   0          8d

NAME                                         STATUS   VOLUME
CAPACITY   ACCESS MODES   STORAGECLASS               AGE
persistentvolumeclaim/my-alertmanager-claim-alertmanager-main-0   Bound    pvc-0d519c4f-
15a5-11ea-baa0-026d231574aa   40Gi       RWO           ocs-storagecluster-ceph-rbd   8d
persistentvolumeclaim/my-alertmanager-claim-alertmanager-main-1   Bound    pvc-
0d5a9825-15a5-11ea-baa0-026d231574aa   40Gi       RWO           ocs-storagecluster-ceph-
rbd   8d
persistentvolumeclaim/my-alertmanager-claim-alertmanager-main-2   Bound    pvc-
0d6413dc-15a5-11ea-baa0-026d231574aa   40Gi       RWO           ocs-storagecluster-ceph-
rbd   8d
persistentvolumeclaim/my-prometheus-claim-prometheus-k8s-0        Bound    pvc-0b7c19b0-
15a5-11ea-baa0-026d231574aa   40Gi       RWO           ocs-storagecluster-ceph-rbd   8d
persistentvolumeclaim/my-prometheus-claim-prometheus-k8s-1        Bound    pvc-0b8aed3f-
15a5-11ea-baa0-026d231574aa   40Gi       RWO           ocs-storagecluster-ceph-rbd   8d
```

2. Edit the monitoring **configmap**.

```
$ oc -n openshift-monitoring edit configmap cluster-monitoring-config
```

Remove any **config** sections that reference the OpenShift Data Foundation storage classes as shown in the following example and save it.

Before editing

```
.
.
.
apiVersion: v1
data:
  config.yaml: |
    alertmanagerMain:
      volumeClaimTemplate:
        metadata:
          name: my-alertmanager-claim
        spec:
          resources:
            requests:
              storage: 40Gi
          storageClassName: ocs-storagecluster-ceph-rbd
    prometheusK8s:
      volumeClaimTemplate:
        metadata:
          name: my-prometheus-claim
        spec:
          resources:
            requests:
              storage: 40Gi
          storageClassName: ocs-storagecluster-ceph-rbd
kind: ConfigMap
metadata:
  creationTimestamp: "2019-12-02T07:47:29Z"
  name: cluster-monitoring-config
  namespace: openshift-monitoring
  resourceVersion: "22110"
  selfLink: /api/v1/namespaces/openshift-monitoring/configmaps/cluster-monitoring-config
  uid: fd6d988b-14d7-11ea-84ff-066035b9efa8
.
.
.
```

After editing

```
.
.
.
apiVersion: v1
data:
  config.yaml: |
kind: ConfigMap
metadata:
  creationTimestamp: "2019-11-21T13:07:05Z"
  name: cluster-monitoring-config
  namespace: openshift-monitoring
  resourceVersion: "404352"
  selfLink: /api/v1/namespaces/openshift-monitoring/configmaps/cluster-monitoring-config
  uid: d12c796a-0c5f-11ea-9832-063cd735b81c
.
.
.
```

In this example, **alertmanagerMain** and **prometheusK8s** monitoring components are using the OpenShift Data Foundation PVCs.

3. Delete the relevant PVCs. Make sure you delete all the PVCs that are consuming the storage classes.

```
$ oc delete -n openshift-monitoring pvc <pvc-name> --wait=true --timeout=5m
```

<pvc-name>

Is the name of the PVC

6.3. REMOVING OPENSHIFT CONTAINER PLATFORM REGISTRY FROM OPENSHIFT DATA FOUNDATION

Use this section to clean up the OpenShift Container Platform registry from OpenShift Data Foundation. If you want to configure an alternative storage, see Image registry.

The Persistent Volume Claims (PVCs) that are created as a part of configuring the OpenShift Container Platform registry are in the **openshift-image-registry** namespace.

Prerequisites

- The image registry must have been configured to use an OpenShift Data Foundation PVC.

Procedure

1. Edit the **configs.imageregistry.operator.openshift.io** object and remove the content in the **storage** section.

```
$ oc edit configs.imageregistry.operator.openshift.io
```

Before editing

```
.
.
.

storage:
    pvc:
        claim: registry-cephfs-rwx-pvc
.
.
.
```

After editing

```
.
.
.
storage:
  emptyDir: {}
.
.
.
```

In this example, the PVC is called **registry-cephfs-rwx-pvc**, which is now safe to delete.

2. Delete the PVC.

```
$ oc delete pvc <pvc-name> -n openshift-image-registry --wait=true --timeout=5m
```

<pvc-name>

Is the name of the PVC

6.4. REMOVING THE CLUSTER LOGGING OPERATOR FROM OPENSHIFT DATA FOUNDATION

Use this section to clean up the cluster logging operator from OpenShift Data Foundation.

The Persistent Volume Claims (PVCs) that are created as a part of configuring the cluster logging operator are in the **openshift-logging** namespace.

Prerequisites

- The cluster logging instance should have been configured to use the OpenShift Data Foundation PVCs.

Procedure

1. Remove the **ClusterLogging** instance in the namespace.

```
$ oc delete clusterlogging instance -n openshift-logging --wait=true --timeout=5m
```

The PVCs in the **openshift-logging** namespace are now safe to delete.

2. Delete the PVCs.

```
$ oc delete pvc <pvc-name> -n openshift-logging --wait=true --timeout=5m
```

<pvc-name>

Is the name of the PVC